```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m14.sh
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ bash m14.sh
m14.sh: line 6: syntax error near unexpected token `('
m14.sh: line 6: `int main()'
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m15.sh
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ bash m15.sh
m15.sh: line 6: syntax error near unexpected token `('
m15.sh: line 6: `int main()'
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m16.sh
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ bash m16.sh
m16.sh: line 9: syntax error near unexpected token `('
m16.sh: line 9: `int is_executable(const char *path)'
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```c
    pid_t pid;
    int status;

    printf("=========================================\n");
    printf("       CHILD PROCESS WITH ARGUMENTS\n");
    printf("=========================================\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());

        /*
         * Execute echo with multiple arguments
         */
        execlp(
            "echo",
            "echo",
            "Hello",
            "Linux",
            "Process",
            "Execution",
            NULL
        );

        /*
         * Error handling
         */
        perror("execlp");
        exit(EXIT_FAILURE);
    }
    else
    {
        printf("Parent PID: %d\n", getpid());

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
```

```
        /*
         * Error handling
         */
        perror("execlp");
        exit(EXIT_FAILURE);
    }
    else
    {
        printf("Parent PID: %d\n", getpid());

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                   WEXITSTATUS(status));
        }

        printf("Parent process finished.\n");
    }

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```c
{
    pid_t child1, child2;
    int status;

    printf("=============================================\n");
    printf(" PROCESS SYNCHRONIZATION USING WAITPID()\n");
    printf("=============================================\n");

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        printf("Child 1 started. PID = %d\n", getpid());

        sleep(2);

        printf("Child 1 completed.\n");

        exit(10);
    }

    child2 = fork();

    if (child2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child2 == 0)
    {
        printf("Child 2 started. PID = %d\n", getpid());

        sleep(4);

        printf("Child 2 completed.\n");

        exit(20);
    }
```

```
    }

    if (child2 == 0)
    {
        printf("Child 2 started. PID = %d\n", getpid());

        sleep(4);

        printf("Child 2 completed.\n");

        exit(20);
    }

    printf("\nParent PID = %d\n", getpid());

    printf("\nWaiting for Child 1...\n");

    if (waitpid(child1, &status, 0) == -1)
    {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status))
    {
        printf("Child 1 exited with status: %d\n",
               WEXITSTATUS(status));
    }

    printf("\nWaiting for Child 2...\n");

    if (waitpid(child2, &status, 0) == -1)
    {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status))
    {
        printf("Child 2 exited with status: %d\n",
               WEXITSTATUS(status));
    }

    printf("\nAll child processes completed.\n");
    printf("Parent process completed.\n");
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#define MAX_PATH_LENGTH 4096

int is_executable(const char *path)
{
    struct stat file_info;

    if (stat(path, &file_info) != 0)
        return 0;

    if (!S_ISREG(file_info.st_mode))
        return 0;

    if (access(path, X_OK) == 0)
        return 1;

    return 0;
}

void find_command(const char *command)
{
    char *path;
    char *path_copy;
    char *directory;

    char full_path[MAX_PATH_LENGTH];

    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return;
    }

    path_copy = strdup(path);

    if (path_copy == NULL)
```

```
    char *path;
    char *path_copy;
    char *directory;

    char full_path[MAX_PATH_LENGTH];

    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return;
    }

    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        perror("strdup");
        return;
    }

    printf("\nPATH variable:\n");
    printf("%s\n", path);

    printf("\nSearching for command: %s\n", command);

    directory = strtok(path_copy, ":");

    while (directory != NULL)
    {
        snprintf(
            full_path,
            sizeof(full_path),
            "%s/%s",
            directory,
            command
        );

        printf("\nChecking: %s", full_path);

        if (is_executable(full_path))
        {
            printf("  --> FOUND\n");
            printf("\nExecutable location:\n%s\n",
                   full_path);
```

```
  {
      snprintf(
          full_path,
          sizeof(full_path),
          "%s/%s",
          directory,
          command
      );

      printf("\nChecking: %s", full_path);

      if (is_executable(full_path))
      {
          printf("  --> FOUND\n");
          printf("\nExecutable location:\n%s\n",
                 full_path);

          free(path_copy);
          return;
      }

      printf("  --> Not executable/not found");

      directory = strtok(NULL, ":");
  }

  printf("\n\nCommand '%s' not found in PATH.\n",
         command);

  free(path_copy);


t main(int argc, char *argv[])

  printf("============================================\n");
  printf("       COMMAND PATH RESOLUTION\n");
  printf("============================================\n");

  if (argc != 2)
  {
      printf("Usage: %s <command>\n", argv[0]);
      printf("Example: %s ls\n", argv[0]);
      return 1;
  }

  find_command(argv[1]);
```